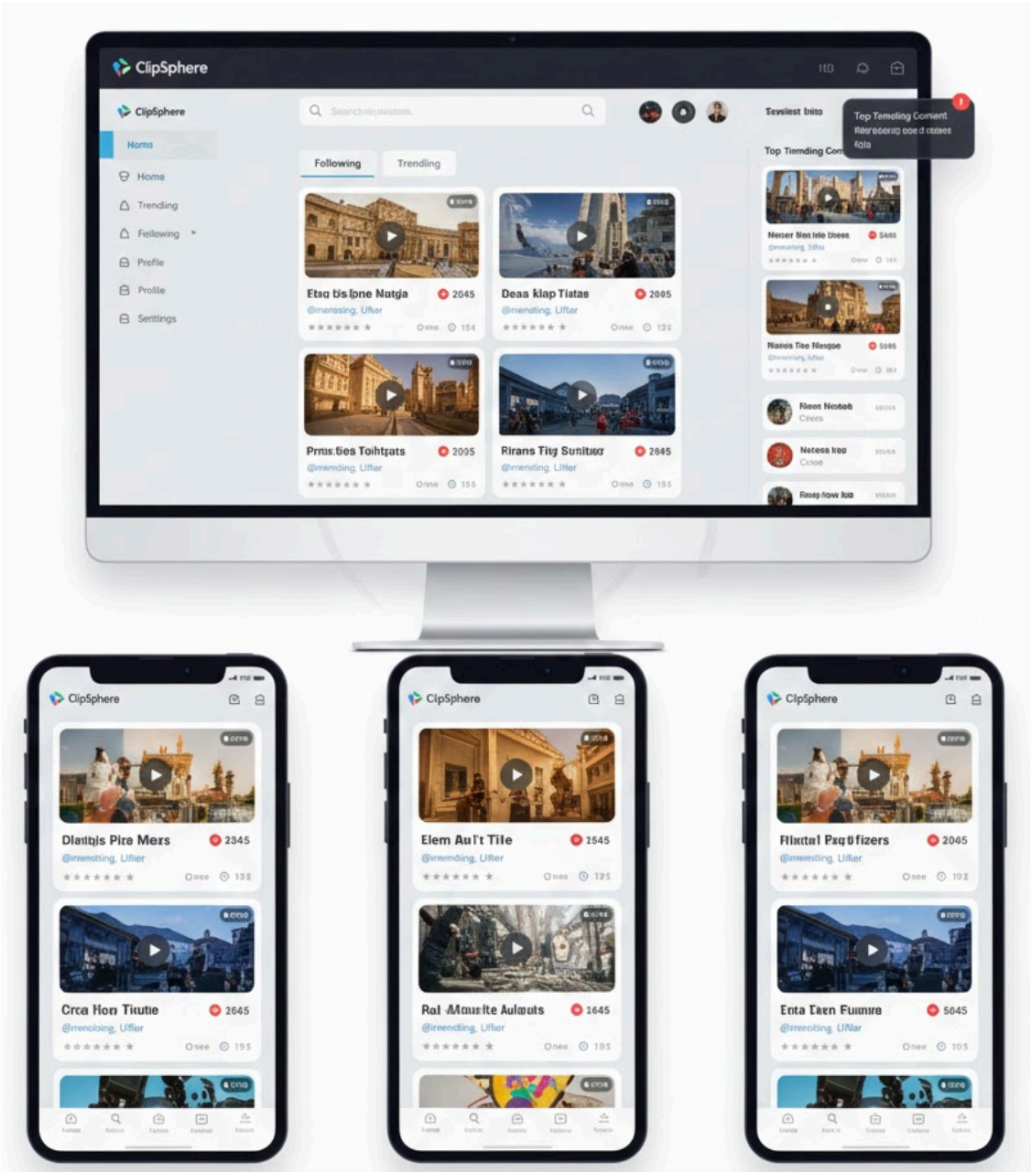
Zewail City of Science and Technology

Course Project (20% of Final Grade)
Team Size: 3-4 Students
Stack: (Next.js, Express, MongoDB, MinIO, Redis, Nginx)
ClipSphere– Short Video Social Platform

Objective: ClipSphere is a high-performance, short-video social platform designed as a comprehensive full-stack ecosystem within a local development environment. The architecture centers on a robust Express and MongoDB backend, integrated with a responsive Next.js frontend styled via Tailwind CSS for a seamless user experience. The infrastructure leverages MinIO for S3-compatible object storage and Nginx as a reverse proxy to handle request routing and security across the services. Technical capabilities include real-time event streaming with Socket.io, integrated financial transactions using Stripe, and background job processing via Redis and Bull. The entire system is orchestrated through Docker, ensuring a cohesive and modular application structure from initial API design to final environment configuration.

Project Phases Timeline

Project Phases	Deliverables	Grade	Deadline
Team Formation	Team size 3-4	**One grade penalty if not submitted on time**	Thursday 26/2/2026
Phase 1	Backend Foundations & Security	**6**	Wednesday 18/3/2026
Phase 2	Next.js Integration & Local Media Pipeline	**4.5**	Sunday 25/3/2026
Phase 3	Real-Time System & Monetization	**4.5**	Sunday 12/4/2026
Phase 4	Scalability, Nginx & Local DevOps	**5**	Sunday 10/5/2026



This project is an opportunity to build a professional piece for your portfolio, something you can truly be proud to feature on your resume. Something to add to your GitHub repo when applying for jobs or internships. Your creativity is highly encouraged; feel free to put your own unique spin on the project's UI and Backend features within the bounds of the requirements.

Phase 1 – Backend Foundations & Security (Details Added)

Build the backend infrastructure and core user system, focusing on data integrity and security protocols. Students will use Postman Client to verify all logic locally before any frontend exists.

Required Features:

1. Foundational System Structure

- **Clean Architecture:** Implementation of a "Three-Layer Architecture" separating the entry point (Routes), the orchestration (Controllers), and the business logic (Services).
- **Local Environment Control:** Secure management of local variables using .env, including PORT, MONGODB_URI, and JWT_SECRET.
- **Global Error Handling:** A centralized middleware to catch all async errors, ensuring the server returns consistent JSON error responses instead of crashing or leaking stack traces.
- **Request Logging:** Integration of Morgan or a custom Winston middleware to log every incoming local request (Method, URL, Status) for easier debugging.
- **Security middleware:** integration of express-mongo-sanitize to prevent NoSQL injection attacks during authentication and data updates.

For this section, below are the expected endpoints ensure the server and middleware are functioning correctly:

Method	Endpoint	Description	Middleware/Logic
GET	/health	Basic heartbeat to check if server is running.	Public
GET	/api/v1/admin/health	**(From Sec 7)** Detailed system health (Uptime, DB connection).	protect + restrictTo(admin)

2. Secure Identity Management

- **User Registration: Endpoint to create new accounts with unique email validation.**
- **Input validation:** implementation of schema-based validation using Zod library for all incoming post/patch requests to ensure data integrity before it reaches the database.
- **Password Security:** One-way hashing using Bcrypt with a salt factor of 10 to ensure user credentials are never stored in plain text.
- **Stateless Authentication:** Implementation of JWT (JSON Web Tokens) with a specific expiration time (e.g., 24h). The token should be returned upon successful login and verified via a "protect" middleware (Authentication: "Who are you?") for private routes.
- **Profile Management:** Endpoints to retrieve current user data and update fields. This includes a metadata-only endpoint for avatars to store the MinIO object name before the storage system is fully active.

3. Permission Levels & Authorization (RBAC)

- **Role Definition:** Implementation of a "role" field in the User schema with values: user and admin.
- **Authorization Middleware:** Custom middleware to restrict endpoints based on roles (e.g., only admins can delete other users' videos; only owners can edit their own profile).
- **Authorization (RBAC):** Middleware to restrict specific endpoints (like "Delete any video" or "View all users") based on a user role field (Standard vs. Admin) in the MongoDB document.
- **Profile Management:** Endpoints to retrieve current user data (GET /me) and update fields (PATCH /updateMe), including metadata-only placeholders for future MinIO avatar links.

<u>Permission Levels & Data Ownership</u>

A. The User Role (Standard Ownership)

- **CRUD Operations:** As a standard user, you have full CRUD (Create, Read, Update, Delete) permissions over your **own** data only.
- **Ownership Verification:** Every "Update" or "Delete" request must pass through an **Ownership Middleware**. This logic compares the req.user.id (from the JWT) with the video.ownerId (from the database). If they don't match, the request is rejected with a 403 Forbidden status.

B. The Admin Role (Platform Oversight)

- **Global Delete Permissions:** Admins are granted "Superuser" status. They can bypass the ownership check specifically for the **Delete** action. This allows them to remove community-violating content without being the video's owner.
- **Restricted Edit Permissions:** Even as an Admin, they should generally **not** be able to "Update" your data (like changing your password or bio), ensuring user privacy is maintained.

For Sections 2 and 3, Secure Identity & RBAC, below are expected endpoints:

Method	Endpoint	Description	Middleware/Logic
POST	/api/v1/auth/register	Creates a new user account.	**Zod Validation**, **Bcrypt Hashing**
POST	/api/v1/auth/login	Exchanges credentials for a JWT.	**Bcrypt Compare**

GET	/api/v1/users/me	Retrieves the logged-in user's data.	protect (Authentication)
PATCH	/api/v1/users/updateMe	Updates username, bio, or avatar key.	protect + **Zod Validation**
GET	/api/v1/users/:id	View a specific user's public profile.	Public

4. Database Collections & Schema Design

- Odm integration: mandatory use of the mongoose library to define schemas, enforce validation, and manage database connectivity.
- **User Collection:** Fields for username, email, hashed password, role, bio, and avatar key. Enforce unique indexes on email and username fields at the database level. Add an active boolean field (default: true) to support admin "soft deletes" and an accountstatus field for moderation.
- **Video Collection:** Fields for title, description, owner (ObjectId reference linked to 'user' model), videoURL (key), duration, and timestamps. Add a viewscount field (default: 0) and a status field (e.g., 'public', 'private', 'flagged') for admin oversight.
- **Review Collection:** Fields for rating (1–5), comment text, user (ObjectId), and video (ObjectId). Implement a logic constraint or compound index to prevent a user from submitting multiple reviews on the same video. Use mongoose min and max validators on the rating field to strictly enforce the 1–5 star range.
- **Followers Collection:** Tracking relationships between users (follower vs. following). Add a compound unique index on followerid and followingid to prevent duplicate relationships. Implement pre-save hooks to prevent the "logic bug" where a user attempts to follow themselves.

5. Social Graph & Preferences

- **Follow Logic:** A dedicated schema or sub-document structure in MongoDB to manage relationships. This must prevent a user from following themselves and ensure that unfollowing correctly removes the reference from the database.

- **Granular Notification Schema:** A nested object within the User model dedicated to preferences. It must include boolean toggles for:
  - In-app alerts: (Followers, Comments, Likes, Tips)
  - Email alerts: (Followers, Comments, Likes, Tips)
- **Event Tracking Logic:** Backend logic that checks these preferences during a social action to determine if a notification record should be created or an email should be queued later.

For the Social Graph & Preferences, below are expected endpoints:

Method	Endpoint	Description	Middleware/Logic
POST	/api/v1/users/:id/follow	Follow a user.	protect + **Self-follow check hook**
DELETE	/api/v1/users/:id/unfollow	Unfollow a user.	protect
GET	/api/v1/users/:id/followers	List all users following this account.	Public
GET	/api/v1/users/:id/following	List all accounts this user follows.	Public
PATCH	/api/v1/users/preferences	Update In-app/Email alert toggles.	protect (Auth)

6. Media & Review Logic

- Video Metadata Schema: Initial design for the Video collection, including title, description, duration, and owner ID.
- Duration Constraints: Logic to store and validate that the video length does not exceed 300 seconds (5 minutes).

- Review System Foundation: Schema for video reviews (1-5 star ratings and text) linked to both the User and the Video.

For the Media & Review Logic, below are expected endpoints:

Method	Endpoint	Description	Middleware/Logic
POST	/api/v1/videos	Create metadata (Placeholder for upload).	protect + **Duration < 300s logic**
GET	/api/v1/videos	Feed: List all public videos.	Public
PATCH	/api/v1/videos/:id	Update video title or description.	protect + **Ownership Middleware**
DELETE	/api/v1/videos/:id	Remove video (Owner or Admin).	protect + **Ownership/Admin Logic**
POST	/api/v1/videos/:id/reviews	Submit a 1-5 star review.	protect + **Unique review index**

7: Admin Oversight & Analytics

- **Platform Statistics Endpoint:** A dedicated GET /api/v1/admin/stats route (protected by Admin-only middleware) that uses MongoDB Aggregation Pipelines to return:
  - Total number of users and videos.
  - Total tips processed (related to Phase3).
  - The "Most Active" users of the week.
- **User Management (The "Ban Hammer"):** An endpoint to "Soft Delete" or deactivate a user account (PATCH /api/v1/admin/users/:id/status). This updates the active: false field we added to the schema.
- **Content Moderation Queue:** A route to fetch all videos flagged with low review scores or specifically reported by users.

- **System Health Check:** A simple GET /api/v1/admin/health that returns server uptime, memory usage, and database connection status.

For the Admin Oversight Analytics section, below are expected outlines:

Method	Endpoint	Description	Middleware/Logic
GET	/api/v1/admin/stats	Site analytics (Total users/videos/tips).	protect + restrictTo(admin)
PATCH	/api/v1/admin/users/:id/status	Deactivate/Ban user (Soft delete).	protect + restrictTo(admin)
GET	/api/v1/admin/moderation	Queue of flagged/low-rated content.	protect + restrictTo(admin)

8. Interactive API Documentation (Swagger)

- **OpenAPI Specification:** Integration of swagger-jsdoc and swagger-ui-express to automatically generate documentation from code comments.
- **Endpoint Visualization:** All routes (Auth, Users, Videos, Admin) must be documented with expected request bodies, query parameters, and potential error responses (400, 401, 403, 404).
- **Security Definition:** Configuration of Swagger to support **JWT Bearer** Authentication, allowing the "Try it out" feature to work for protected routes.
- **Access Point:** The documentation must be served at a dedicated local route (e.g., http://localhost:5000/api-docs).

For swagger, expected endpoint below:

Method	Endpoint	Description	Access
GET	/api-docs	Interactive Swagger UI dashboard.	Public

Deliverables

- Everything mentioned above
- GitHub repository with organized folders for routes, controllers, services, models, and middleware.
- Organized folder “models/” directory containing Mongoose Schemas for all collections.
- **Interactive documentation:** integration of swagger-jsdoc and swagger-ui-express. the api must be accessible via /api-docs, showing all schemas, parameters, and authentication headers required for each endpoint.
- **Functional API:** A fully working backend that passes local integration tests for Registration, Login, and RBAC.
- **API Documentation:** A Postman Collection including Environment Variables for base URLs, Scripts for automated token management, and Basic Tests for every endpoint.
- **Database ER Diagram:** A visual map of the MongoDB collections highlighting the relationships between Users, Videos, and Reviews.

Phase 2 – Next.js Integration & Local Media Pipeline

[Dr. Rakha may apply changes, add more details, and requirements to this Phase later as more course materials are covered.]

Objectives: Launch the Next.js frontend and configure a local MinIO instance via Docker to simulate cloud object storage. Establish a strict media pipeline that validates content before it is stored or served to the discovery feeds.

Required Features

1. Next.js & Tailwind Setup

- **Frontend Scaffolding:** Next.js (App Router) initialization on localhost:3000, utilizing a directory structure that separates components, hooks, and services.
- **Responsive Design:** Implementation of a Mobile-first navigation system and global layout wrappers using Tailwind CSS utility classes for seamless screen transitions.
- **Client-Side Security:** Next.js Middleware configuration to intercept requests; it validates JWT cookies to protect private routes like /upload or /settings. Sync auth state, implement a useAuth hook that consumes the /api/v1/users/me endpoint defined in phase 1 to persist user sessions across page refreshes.
- **State Management:** Use of React hooks (useState, useEffect) or lightweight libraries to manage UI states such as loading spinners and error modals.

2. Local MinIO Infrastructure

- **Dockerized Storage:** Deployment of MinIO via Docker Desktop, enabling access to the storage console on localhost:9001 for manual bucket management.
- **S3 SDK Integration:** Configuring the Express backend with the AWS S3 SDK to point to the local MinIO endpoint instead of the public cloud.
- **Security via Presigned URLs:** Backend logic to generate temporary, time-limited Presigned URLs; this ensures video files remain private and are only accessible through the application.

3. Content Validation & Storage

- **Duration Enforcement:** Integration of fluent-ffmpeg on the backend to probe metadata of uploaded files; the system must strictly reject and delete any video exceeding 300 seconds (5 minutes).
- **File Type & Size Filtering:** Multer middleware configuration to validate MIME types (e.g., video/mp4) and restrict file sizes to prevent local disk exhaustion.
- **Object Storage Mapping:** Logic to store the unique MinIO "Object Key" in MongoDB, linking the physical file to the video’s metadata and owner profile. Atomic updates, logic to ensure that the minio "object key" is only saved to the MongoDB video collection after the S3 upload is confirmed successful, preventing "orphaned" database records.

4. Responsive Discovery & Scrollable Feeds

- **Grid System:** Tailwind-based grid layout adjusting dynamically: 1 column for mobile, 2 for tablets, and 3-4 for desktop displays.
- **Paginated Discovery:** API implementation of "Limit" and "Skip" logic to serve data in chunks, preventing the browser from crashing when loading large numbers of videos.
- **Infinite Scroll:** Frontend integration of Intersection Observer API to detect when a user reaches the bottom of the feed and automatically trigger the next page fetch.
- **Feed Logic:** Algorithmic separation into "Following" (filtering by followed user IDs) and "Trending" (sorting by average review scores and recent engagement). For aggregation

consumption, the "trending" feed must consume the aggregation pipelines created in phase 1, section 7, to sort content by recent engagement and average rating.

5. Video Interaction & Review UI

- **Player Integration:** Implementation of a responsive video player (HTML5 or React-Player) that supports custom controls and displays duration overlays.
- **Review Interface:** Tailwind-styled star-rating system and comment fields positioned directly below the player for immediate user feedback. For the dynamic ownership ui, the ui must hide "edit" and "delete" buttons on videos and reviews unless the logged-in user id matches the ownerid returned by the api (respecting phase 1 ownership middleware).
- **Dynamic Feedback:** Real-time UI updates for Like/Unlike actions and Review submissions to improve perceived app speed.

6. Engagement & Email Automation

- **Component Logic:** Creation of reusable Like, Comment, and Share components with optimistic UI updates.
- **Email System:** Nodemailer integration using HTML templates for Welcome emails and "New Engagement" alerts.
- **Preference Check:** Backend service that verifies the recipient's MongoDB "Notification Preferences" before executing the email send command.

Admin Dashboard UI

- **Admin Panel UI:** creation of a protected /admin route that renders the statistics (total users/videos) and system health metrics from the phase 1 admin endpoints. Dashboard accessible only to accounts with the 'admin' role. It must render data from the phase 1 statistics endpoint (total users, videos, and system health).]

Deliverables:

- Everything mentioned above
- **Complete Frontend:** A responsive Next.js web application running locally and fully connected to the Express API.
- **Media Pipeline:** A working video upload system that successfully filters by duration and stores files in local MinIO buckets.
- **Interactive Feed:** A scrollable discovery page where users can view videos, read reviews, and submit their own ratings.
- **Notification Engine:** An automated email service that triggers based on social actions and honors user-defined settings.

Phase 3 – Real-Time System & Monetization

[Dr. Rakha may apply changes, add more details, and requirements to this Phase later as more course materials are covered.]

Objectives: Implement real-time updates using WebSockets and integrate Stripe's developer sandbox.

Required Features:

1. Socket.io Real-Time Layer

- **Personalized Socket Rooms:** Upon connection, the backend uses the authenticated User ID to place each user into a private "User Room" (socket.join(userId)). This ensures notifications are routed to the specific recipient and not broadcasted to the whole site.
- **The "Live Like" Trigger:** When a user clicks the "Like" button, the backend Like Controller identifies the **Video Owner** and immediately emits a new-like event to that owner's specific room.
- **Notification Bubble UI:** A floating, Tailwind-styled "toast" bubble that slides in from the top-right corner of the uploader's screen, displaying the liker's username and the video title.
- **Global Engagement Hub:** Real-time badge updates on the navigation bar (e.g., a red dot appearing over the "Activity" icon) that persists until the user views the notification.

2. Stripe Monetization (Test Mode)

- **Creator Tipping System:** Integration of Stripe Checkout for one-time tips. The backend must map the Stripe payment_intent to the correct creator's wallet in MongoDB.
- **Local Webhook Management:** Utilizing the Stripe CLI to listen for checkout.session.completed events and update the database locally.
- **Financial Ledger:** A dedicated Transaction collection to track history, ensuring creators can see their "pending balance" from tips.
- Transaction history and creator balance tracking in MongoDB

3. Security & Validation

- Zod or Joi for input validation on both frontend and backend
- Rate limiting on API and MinIO upload routes
- Helmet.js and CORS policy configuration for local cross-origin requests

4. Advanced UI

- Tailwind glassmorphism for video overlays
- Skeleton loaders for the video feed to improve perceived speed

Deliverables

- Everything mentioned above
- Live Tipping feature using Stripe Test Mode
- Real-time socket-based notification system
- Security-hardened API and frontend

Phase 4 – Scalability, Nginx & Local DevOps

[Dr. Rakha may apply changes, add more details, and requirements to this Phase later as more course materials are covered.]

Objectives Package the entire ecosystem into a multi-container environment (Microservices) where every service runs in its own isolated Docker container. Use Nginx as the central gateway to route traffic to these individual services under a single local domain.

Required Features

1. Multi-Container Orchestration (Docker Compose)

- **Service Isolation:** Every component must run in its own dedicated container:
  - **Container A:** Next.js (Frontend)
  - **Container B:** Express.js (API Backend)
  - **Container C:** MongoDB (Primary Database)
  - **Container D:** MinIO (Object Storage)
  - **Container E:** Redis (Task Queue & Caching)
  - **Container F:** Nginx (Reverse Proxy Gateway)
- **Multi-stage Dockerfiles:** Optimized builds for the frontend and backend to minimize image size and local resource usage.
- **Service Networking:** Creating a virtual Docker network so containers can communicate using internal service names (e.g., the Backend connects to mongodb://database:27017).

2. Local Nginx Reverse Proxy (The Gateway)

- **Unified Entry Point:** Nginx acts as the only container exposed to your local port 80. It handles all incoming traffic and routes it:

    - localhost/ → Next.js Container
    - localhost/api → Express Backend Container
    - localhost/storage → MinIO Container
- **Static Asset Offloading:** Configuring Nginx to serve the frontend's static assets directly, reducing the load on the Next.js container.
- **Local SSL:** Implementing self-signed certificates so the entire local ecosystem runs over HTTPS.

3. Background Tasks & Worker Process (Redis & Bull)

- **Task Queuing:** Using the dedicated Redis container to manage a BullMQ queue for heavy lifting.
- **Worker Isolation:** (Optional but Recommended) Running a separate "Worker" instance of the backend container specifically to process email sending and video duration metadata, ensuring the main API remains responsive to users.
- **Persistence:** Configuring Docker Volumes for MongoDB and MinIO so that data and videos are not lost when containers are stopped or restarted.

4. System Verification & Performance Testing

- **Caching Layer:** Implementing Redis caching for the "Trending" video feed to ensure that frequent requests for the most popular videos do not hit the MongoDB database repeatedly.
- **Bulk Media Testing:** Each team is required to perform a "Stress Test" by uploading multiple videos simultaneously to verify that the 5-minute duration gate, MinIO storage, and background processing all function under load.
- **Technical Documentation:** Creation of a full architecture diagram and comprehensive API documentation (Swagger or Postman).
- **Reflection & Deployment Guide:** A team report summarizing technical hurdles and a step-by-step guide on how to launch the entire system from a fresh docker-compose up command.

Deliverables

- Everything mentioned above
- Full stack running via a single "docker-compose up" command
- Local domain access via Nginx (e.g., http://clipsphere.local)
- Final performance audit and project presentation
- Orchestrated Environment: The full stack running seamlessly via a single Docker command.
- Local Gateway: Application accessible via a single port (80) behind Nginx with verified routing.
- System Audit: Verification of successful multi-video uploads and functioning background email queues.

- Deployment Package: Finalized Git repository containing all Docker configurations and the technical reflection report.

Bonus:

Dr. Rakha may or may not add a bonus to this project, yet to be decided.

Important Rules:

1. For each phase, **if a late submission is permitted, a 25% penalty will be incurred** for each day after the deadline. After two overdue days, no submissions will be accepted.
2. Each phase's deliverables must be uploaded to the **Google Classroom** assigned assignment and pushed to the repository **before the submission deadline**.
3. Teams are encouraged to be from the same lab. Team members can be from different LABS, but NOTE that: It will be your FULL RESPONSIBILITY to attend the evaluation session. Any student who misses the scheduled evaluation session will automatically receive ZERO.
4. The deliverable version uploaded to the **Google Classroom** assignment is the one that will be evaluated. If it's a team project, the team representative can just upload the deliverable for other members. The GitHub version will serve only as a reference to track progress.
5. **Version Control:** All projects must use GitHub to track progress. Commits should reflect contributions from all members. **GitHub is mandatory in the evaluation process starting Phase-1** and must follow the "GitHub Repository Structure" mentioned. Failing to maintain an updated GitHub repository for your projects may result in a penalty of **up to 20%** for the phase under evaluation.
6. **Documentation Requirements**
    a. Each phase must include clear and complete documentation as outlined in the tasks, if any.
7. **Although this is a team project, every member is expected to fully understand all submitted work. TAs may ask any student about any part of the deliverable** (diagrams, code, documentation, etc). Any student unable to explain their project/code may lose marks individually, even if the deliverable was submitted.
8. **Cheating** could result in a negative grade (up to - 10 Course Grades).
9. **Academic Integrity Reminder**: Directly copying or pasting content from AI tools (such as ChatGPT or others) into your course deliverables is considered academic misconduct (Up to cheating). Your submitted work should reflect your own effort, writing style, creativity, and uniqueness. You may use AI tools to support your learning or improve your understanding, but the final content must be genuinely your own. All submissions/deliverables are subject to plagiarism and AI-detection checks. If there is reasonable suspicion of AI-generated or copied content, the lecturer/TA reserves the right to:
    - Assign a penalty for **cheating** if misuse is confirmed.
    - Request additional work or conduct an oral examination to verify authorship and understanding of the submitted material.

10. Each phase submission (when needed) must include **clear documentation** as required.

GitHub Repository & Version Control Requirement

Each team must create and maintain a dedicated GitHub repository for this project. All development work must be committed regularly throughout the project period. The repository will be used to track progress, evaluate development consistency, and verify individual contributions.

Teams are expected to make meaningful, incremental commits that reflect continuous development rather than uploading the entire project at once. Commit messages should be clear and descriptive. The repository must include a well-organized folder structure, source code, assets (where applicable), and a README file explaining how to run the project.

Failure to demonstrate a consistent development history through GitHub commit activity may impact the project evaluation.

Grading Policy

- Understanding deliverables and their details will be evaluated individually for each team member.
- Your project GitHub is part of the evaluation weight.
- All the mentioned requirements in each phase are part of the evaluation criteria weights.

Deliverable Discussion/Response Policy:

- If a student is asked a question during evaluation and does not answer, **5 points will be deducted**.
- If the student fails to answer a second question, **an additional 5 points will be deducted**.
- If the student fails to answer a third question, **the student will receive 0 for the Individual Understanding mark**.
- If the student fails to answer a fourth question, the student will receive 0 for the deliverable itself, regardless of the team's mark

Project Deliverable Documents

All project deliverables should have the following cover page:
Zewail City of Science, Technology and Innovation
University of Science and Technology
School of Computational Sciences and Artificial Intelligence

SWAPD352- Spring 2026

Web Development

<Project Name>

Student#1 <Name> <ID>

Student#2 <Name> <ID>

<Date>